Cubic EOS Software: PVT, H, S and Fugacity of Real Gases

Group members : Sagar Durge (2025CH71059 , Group 2)

Core Objective

The main objective of this software/project is to **predict and analyze how real gases behave under various temperatures and pressures**. While ideal gases follow the simple law PV=nRT, real gases deviate significantly especially at high pressures or low temperatures due to intermolecular forces and the actual volume occupied by gas molecules. This software uses **cubic Equations of State (EOS)**—specifically van der Waals (vdW), Redlich/Kwong (RK), Soave/Redlich/Kwong (SRK), and Peng/Robinson (PR) to accurately calculate:

(a) **PVT Behavior:** The relationship between Pressure (P), Molar Volume (V), and Temperature (T), represented by the compressibility factor Z (Z=PV/RT)

(b) **Residual Properties (H^R, S^R):** How much real thermodynamic properties like enthalpy (H) and entropy (S) deviate from ideal gas behavior.

(c) **Fugacity (f) & Phase Behavior:** The "effective pressure" of a real gas, allowing the software to determine whether a gas will condense into a liquid, remain a vapor, or exist in liquid-vapor equilibrium at a given temperature and pressure (including vapor pressure calculations).

Theory

All four EOS can be represented in a single general cubic equation differing only in the constants $\sigma \in \Omega\Psi$

$$P = \frac{RT}{V-b} - \frac{a(T)}{(V+\varepsilon b)(V+\sigma b)}$$

where the temperature-dependent parameter a(T) and parameter b are given by:

$$a(T) = \Psi \frac{\alpha(T_r),\ R^2 T_c^2}{P_c},\ b\ =\ \Omega \frac{R\,T_c}{P_c}$$

EOS	α (Tr)	σ	ϵ	Ω	ψ	Zc
vdW	1	0	0	1/8	27/64	3/8
RK	$(Tr)^{-1/2}$	1	0	0.08664	042748	1/3
SRK	$[1 + \kappa(1-Tr^{(1/2)})]^2$, $\kappa = 0.480 + 1.574\omega - 0.176\omega^2$	1	0	0.08664	0.42748	1/3
PR	$[1 + \kappa(1 - Tr^{(1/2)})]^2$, $\kappa = 0.37464 + 1.54226\omega - 0.26992\omega^2$	$1+\sqrt{2}$	$1-\sqrt{2}$	0.07780	0.45724	0.307

Now we convert the equation into:

$$P = \frac{RT}{V-b} - \frac{a(T)}{V^2 + ubV + wb^2}$$

where u =$\sigma + \epsilon$ and w = $\sigma\epsilon$ .The EOS parameters (u,w) are defined as (0,0) for vdW, (1,0) for RK and SRK, and (2,-1) for PR.

Defining the non-dimensional parameters A = $\frac{aP}{(RT)^2}$and B = $\frac{bP}{RT}$, the program transforms the equation into a cubic polynomial in terms of the compressibility factor Z = $\frac{PV}{RT}$

$$Z^3 + \alpha Z^2 + \beta Z + \gamma = 0$$

Where:

$$\alpha = -1 - B + uB$$

$$\beta = A + wB^2 - uB - uB^2$$

$$\gamma = -AB - wB^2 - wB^3$$

Instead of numerical iteration, the program solves the cubic equation analytically using **Newton Raphson's Method**. The physically meaningful real roots must satisfy Z > B (ensuring V > b):

- **Single-phase region**: A single real root represents the fluid state.
- **Two-phase subcritical region**: Three real roots exist. The largest root (Z_{max}) represents the **vapour root**, the smallest root (Z_{min}) represents the **liquid root**, and the middle root has no physical meaning and is ignored.
- **Thermodynamic stability**: When selecting the stable phase at a given T and P, the program computes the normalized residual Gibbs free energy$\frac{G^R}{RT} = \ln \quad \phi$ for all valid roots and picks the root yielding the minimum value (the condition for thermodynamic equilibrium at constant T, P).

The thermodynamic departure functions follow from integrals over density:

$$\frac{H^R}{RT} = Z - 1 - T\int_0^\rho \left(\frac{\partial Z}{\partial T}\right)_\rho \frac{d\rho}{\rho}$$

$$\frac{S^R}{R} = \ln Z - T\int_0^\rho \left(\frac{\partial Z}{\partial T}\right)_\rho \frac{d\rho}{\rho} - \int_0^\rho (Z-1)\frac{d\rho}{\rho}$$

Integrating these expressions analytically yields closed-form solutions:

$$I = \frac{1}{\sigma - \varepsilon}\ln\left(\frac{Z + \sigma B}{Z + \varepsilon B}\right) \qquad \left(\text{vdW: } I = \frac{B}{Z}\right)$$

$$\frac{H^R}{RT} = Z - 1 + \left[\frac{d\ln\alpha}{d\ln T_r} - 1\right] qI \qquad \text{where } q = \frac{A}{B} = \frac{a}{bRT}$$

$$\ln \phi = \frac{G^R}{RT} = Z - 1 - \ln(Z - B) - qI \qquad \Rightarrow f = \phi P$$

The temperature derivative $\frac{d \ln \alpha}{d \ln T_r}$ is:

vdW: 0
RK: -1/2
SRK & PR: $\frac{-\kappa\sqrt{T_r}}{1+\kappa(1-\sqrt{T_r})}$

Total property values and changes between states (T1, P1) and (T2, P2) are evaluated by combining ideal-gas heat capacity integrations with state departure functions:

$$\frac{C_p^{\text{ig}}}{R} = A + BT + CT^2 + DT^{-2}$$

$$\Delta S = R \int_{T_1}^{T_2} \frac{C_p^{\text{ig}}}{R} \frac{dT}{T} - R \ln\left(\frac{P_2}{P_1}\right) + S_2^R - S_1^R$$

Absolute Enthalpy (H) and Entropy (S) are reported relative to an ideal gas reference state at T_{ref} = 298.15K and P_{ref} = 1.0 bar:

$$S(T,P) = R \int_{298.15}^{T} \frac{C_p^{\text{ig}}}{R} \frac{dT}{T} - R \ln\left(\frac{P}{1.0}\right) + S^R(T,P)$$

Program structure and algorithms

The program is structured as a modular Python architecture utilizing basic classes , functions and libraries. It relies on standard library modules (math).

Class/dictionary/Function	What it does
GASES (dictionary)	Stores T_c, P_c, ω (Table B.1) and heat-capacity polynomial parameters A, B, C, D for C_p^{ig}/R (Table C.1) for standard gases.
CubicEOSNewtonRaphson	Main calculator class that encapsulates EOS parameters and solving logic
alpha_function	Calculates the temperature-dependent correction factor $\alpha(T_r)$ and kappa parameter κ.
__init__	Initializes the instance and sets generic EOS constants u, w, Ω, Ψ for vdW, RK, SRK, and PR models.
dlnalpha_dlnTr	Computes the derivative $\frac{d\ln\alpha}{d\ln T_r}$ required for residual enthalpy and entropy calculations.
get_a_and_b	Computes attraction parameter $a(T)$ and excluded volume b from critical properties (T_c, P_c, ω).
solve_Z	Builds polynomial coefficients (α, β, γ); finds the largest root using **Newton-Raphson iteration** starting at Cauchy's root bound; performs synthetic division to extract secondary quadratic roots; keeps roots where Z > B.

calculate_state	Selects the vapor root (largest Z), evaluates density integral I, computes residual properties (H^R, S^R,$\ln \phi, f$), and returns a dictionary of physical outputs.

When three real roots exist, Newton-Raphson finds the largest root (Z_{vapor}) by initiating the solver at Cauchy's upper bound $Z_0 = 1.0 + |\alpha| + |\beta| + |\gamma|$. Synthetic division then extracts the quadratic remainder to find secondary roots (Z_{liquid} and the non-physical middle root). Valid physical roots satisfy Z > B.

How Newton-Raphson is Implemented Here:

1. Upper Bound Guess (Z0):

$$Z_0 = 1.0 + |\alpha| + |\beta| + |\gamma|$$

Guarantees the initial guess starts to the right of all roots.

2. Iterative Slope Update:

$$f(Z) = Z^3 + \alpha Z^2 + \beta Z + \gamma$$

$$f'(Z) = 3Z^2 + 2\alpha Z + \beta$$

$$Z_{k+1} = Z_k - \frac{f(Z_k)}{f'(Z_k)}$$

Runs until the difference $|Z_{k+1} - Z_k| < 10^{-12}$

3. Deflation / Synthetic Division:

Once the largest root (Z_1) is found, it divides the cubic by $(Z - Z_1)$to form a quadratic equation $Z^2 + pZ + q = 0$, directly extracting any secondary roots.

Results and validation

The program reproduces every textbook benchmark we tested. For SVNA Example 3.9 (nbutane, R_K 350 K, 9.4573 bar) it gives V_v= 2555.3 and V_l= 133.3 cm³/mol, against 2555 and 133.3 in the book. For SVNA Table 13.7 (n-butane, 500 K, 50 bar):

EOS	Z book	Z program	Book H^R	Program H^R
vdw	0.6608	0.6610	-3937	-3935
RK	0.6850	0.6852	-4505	-4503
SRK	0.7222	0.7224	-4824	-4821
PR	0.6907	0.6909	-4988	-4985

The small differences in the fourth digit come from the book rounding R and intermediate values.

PR vapour pressures from the equal-fugacity condition agree with reference values within 0.3% for propane (300 K), n-butane (350 K) and CO_2 (280 K), and within 0.7% for ammonia (300 K). Water at 373.15 K is off by 5.3% (0.960 vs 1.014 bar) because PR with the standard α function does not represent strongly polar, hydrogen-bonded fluids well beacause of the following reasons :

(a) **Missing Physics (Hydrogen Bonding)**: Standard cubic equations of state assume simple, spherical or slightly non-spherical van der Waals forces. They do not account for strong directional forces like hydrogen bonding or permanent dipole-dipole interactions found in water and alcohols.

(b) **Standard $\alpha(T_r)$ Function**: The standard temperature correction term $\alpha(T_r)$depends only on reduced temperature (T_r) and the acentric factor (ω), which measures how non-spherical a molecule is.

Discussion and conclusions

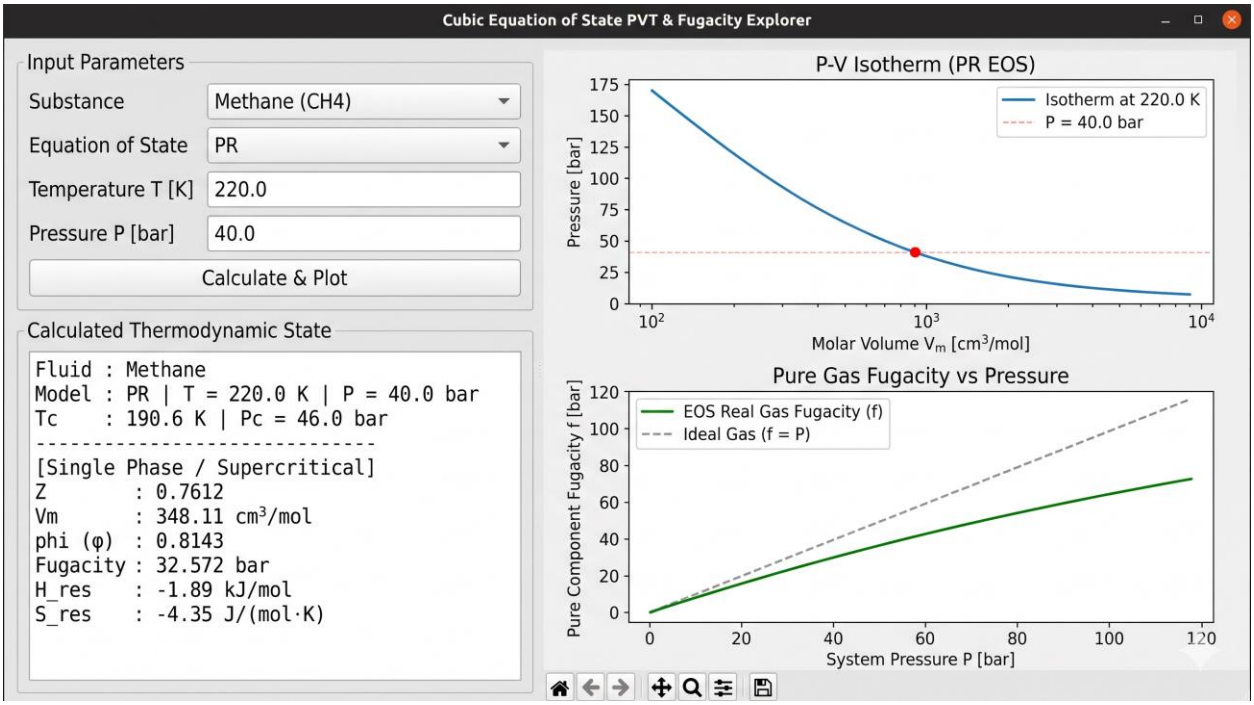
SRK and PR are the equations to use for engineering work on non-polar gases. Both give H^R within about 100 J/mol and S^R within 0.3 J/(mol K) of reference data, while vdW errors are five to eight times larger. RK has the smallest mean Z error in our gas-phase set, but it has no acentric factor, so its vapour pressures and liquid volumes are poor.

Limitations:

(a) **Overprediction of Compressibility (Z)**: Cubic equations of state inherently overpredict the compressibility factor near the critical point ($T \approx T_c, P \approx P_c$).

(b) **Inadequate for Hydrogen Bonding**: The standard temperature dependency function $\alpha(T_r)$ used in PR and SRK relies solely on reduced temperature (T_r) and acentric factor (ω).

(c) **Extrapolation Below** 298.15 K: The ideal-gas heat capacity polynomial coefficients (A, B, C, D) from Table C.1 are strictly valid from .



Software Implementation

The Python application is split into two modules:

1. **Calculation Engine (CubicEOSNewtonRaphson)**: Contains pure fluid properties (T_c, P_c, ω), Newton-Raphson root solver initialization using Cauchy's upper bound with synthetic root extraction, and thermodynamic functions for $Z, V_m, \phi, f, H^R, and\ S^R$.
2. **Desktop User Interface (Tkinter + Matplotlib)**: Allows selecting fluids ($CO_2, CH_4, C_{2H_6}, N_2, H_{2O}$) and models (VDW, RK, SRK, PR) with temperature/pressure inputs. It generates live P--V_m isotherms and fugacity curves ($fvs.P$).

Reference: J. M. Smith, H. C. Van Ness, M. M. Abbott and M. T. Swihart, Introduction to Chemical Engineering Thermodynamics, 8th ed., McGraw-Hill, 2018: Table 3.1, Secs. 3.6, 6.4, 10.5, 13.6, 13.7, Tables B.1 and C.1. Reference fluid data: I. H. Bell et al., Ind. Eng. Chem. Res. 53 (2014) 2498 (CoolProp).